

BioGen NL Agent – Phase 1, Phase 2 and Phase 3 Report

Data Parsing, Indexing, Search, Factually Grounded RAG and Deep Research Agent

Alexandre Santos 72970
Miguel Mestre 73018
Natural Language Processing and Search
2025/2026

June 4, 2026

1 Introduction

In high-stakes settings such as biomedical question answering, factual grounding is essential. This report covers the three phases of the BioGen NL Agent. Phase 1 builds and evaluates the retrieval layer over a PubMed corpus. Phase 2 extends this retrieval system into a factually grounded RAG pipeline with sentence-level evidence selection, citation-bearing answer generation, and LLM-as-a-judge evaluation. Phase 3 further extends the system into a biomedical Deep Research Agent that decomposes a patient question into subtopics, retrieves and aggregates evidence for each subtopic, and synthesizes a structured long-form report grounded in PubMed evidence.

2 BioGen

2.1 Data Parsing, Indexing and Search

2.1.1 Corpus and Topic Data

Each PubMed record was parsed into three fields: `pmid`, `title`, and `abstract`. Each topic specifies an information need via `topic`, `question`, and `narrative`, which were concatenated into a single query string for retrieval.

2.1.2 Relevance Judgments

Relevance judgments were derived from the provided submission annotations. For each question, cited PMIDs in accurate system answers were inspected. Citations labeled as *supporting* were assigned relevance score 2, citations labeled as *neutral* were assigned relevance score 1, and citations labeled as *contradicting* or *not relevant* assigned relevance score 0. This produced graded relevance judgments suitable for NDCG evaluation, as well as binary relevance sets for precision, recall, MAP, and MRR.

2.1.3 Indexing in OpenSearch

The retrieval system was implemented in OpenSearch. Each document was indexed with identity fields (`pmid`, `title`, `abstract`), three separate lexical content fields each using a different similarity function (BM25, JM, Dirichlet), and a dense `embedding` field for KNN search. Separating the lexical fields enables a fair comparison over the same document text.

2.1.4 Retrieval Models

We implemented four retrieval strategies: **BM25**, **Language Model with Jelinek-Mercer smoothing (JM)**, **Language Model with Dirichlet smoothing (LMD)**, and **KNN** dense retrieval. For KNN, documents and queries were encoded into dense vectors using a sentence embedding model and retrieved by nearest-neighbor search in OpenSearch.

2.2 LLM Augmented Generation Phase 2

2.2.1 Overview

In Phase 2, the retrieval component developed in Phase 1 was extended into a factually grounded retrieval-augmented generation pipeline. The objective of this phase is to generate a single biomedical answer for each topic using only evidence extracted from the retrieved PubMed documents.

Since KNN was the strongest retrieval method in Phase 1, the Phase 2 system uses the KNN test run as input. For each query, the top 3 retrieved documents are selected, and up to 3 reference sentences are extracted per document.

2.2.2 Reference Sentence Selection

Each abstract was split into sentences; very short fragments were discarded. Each sentence was scored by pairing it with the query using `ncbi/MedCPT-Cross-Encoder`, and the top 3 sentences per article were kept as reference evidence. Results were saved to `phase2_reference_sentences.json`.

2.2.3 Answer Generation

The selected reference sentences were passed to an instruction-tuned biomedical decoder model, `google/medgemma-4b-it`. The prompt instructed the model to answer only using the provided reference sentences and to avoid adding external biomedical knowledge.

The prompt constrained the model to at most 250 words, required every sentence to carry at least one citation in `[PMID:id]` format (max three per sentence, only from the provided reference set), and instructed the model to acknowledge insufficient evidence rather than hallucinate.

The generated answers were saved to `phase2_generated_answers.json`.

2.2.4 LLM-as-a-Judge Evaluation

The final stage used GPT-4o through the IAedu API as an LLM-as-a-judge. Two types of judgments were collected.

Each reference sentence was labeled for alignment (**Required**, **Borderline**, **Unnecessary**, **Inappropriate**), and each generated answer was labeled for entailment (**Supported**, **Partially Supported**, **Unsupported**).

The judge results were saved to `phase2_judge_results.json`.

Prompt calibration. The specification prompts were adjusted before running the full evaluation: the judge’s role was changed from “expert annotator” to “expert biomedical annotator”, and explicit input labels (`Question:`, `Complete answer:`, `Reference sentences:`) were added to prevent the model from confusing which section it was evaluating. A small-sample test confirmed that labels matched expectations before the full run.

2.2.5 BERT Embeddings and Self-Attention Exercise

In addition to the RAG pipeline, we implemented the required BERT visualization exercise using `bert-base-uncased`. Three analyses were performed.

Positional embeddings. To isolate positional information, we built a 200-token sequence by repeating the word “medicine” and projected the positional embeddings to 2D with PCA. The PCA plot traces a smooth spiral (Figure 1, left), showing that nearby positions cluster together and distance grows gradually along the curve. To confirm this, we also computed the full 200×200 pairwise distance matrix (right). The dark diagonal shows that neighboring positions are similar; the bright yellow corners show that positions far apart in the sequence are maximally different. The smooth color gradient between them confirms there are no sudden jumps — distance grows steadily with sequence separation.

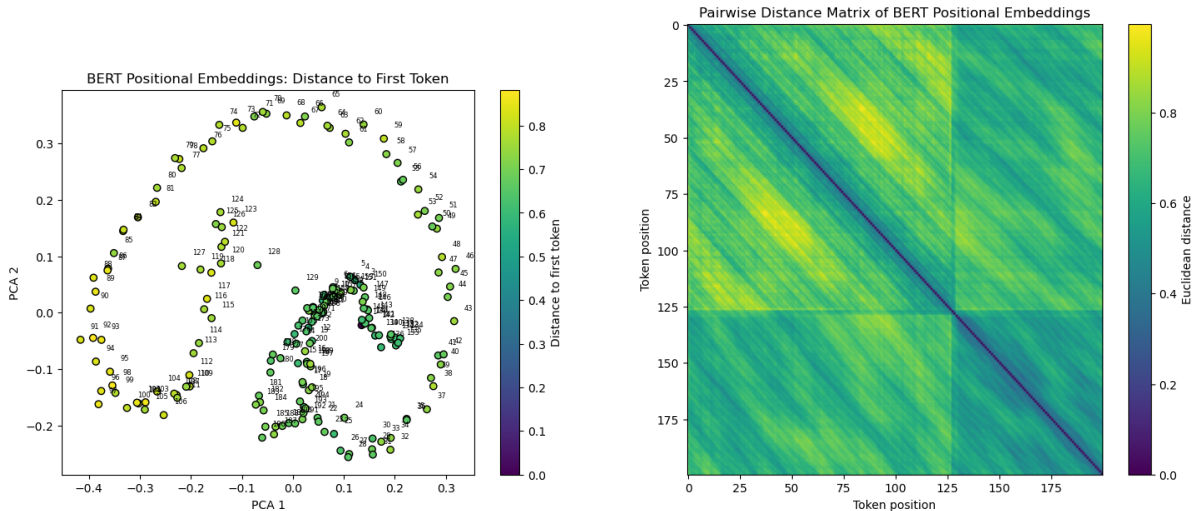


Figure 1: Left: PCA of BERT positional embeddings colored by distance to position 1. Right: pairwise distance matrix. Both confirm continuous, ordered position encoding.

Contextual embeddings. We fed BERT a query-passage pair: “How can sleep apnea be improved?” paired with “Sleep apnea can be improved by weight loss, avoiding alcohol, using positional therapy, and applying continuous positive airway pressure.” The two sentences are joined by a [SEP] token, giving a 37-token input. We extracted the token representations at each of the 12 transformer layers and projected them with PCA.

Figure 2 shows each layer side by side. At layer 0 tokens are tightly packed, since the model has not yet processed any context. By layers 9–11 the spread is roughly ten times larger, and tokens that appear in both sentences (such as *sleep* and *apnea*) have diverged into distinct positions — shaped by their different surrounding context in the query versus the passage. This is how the cross-encoder builds a representation that reflects meaning, not just word identity.

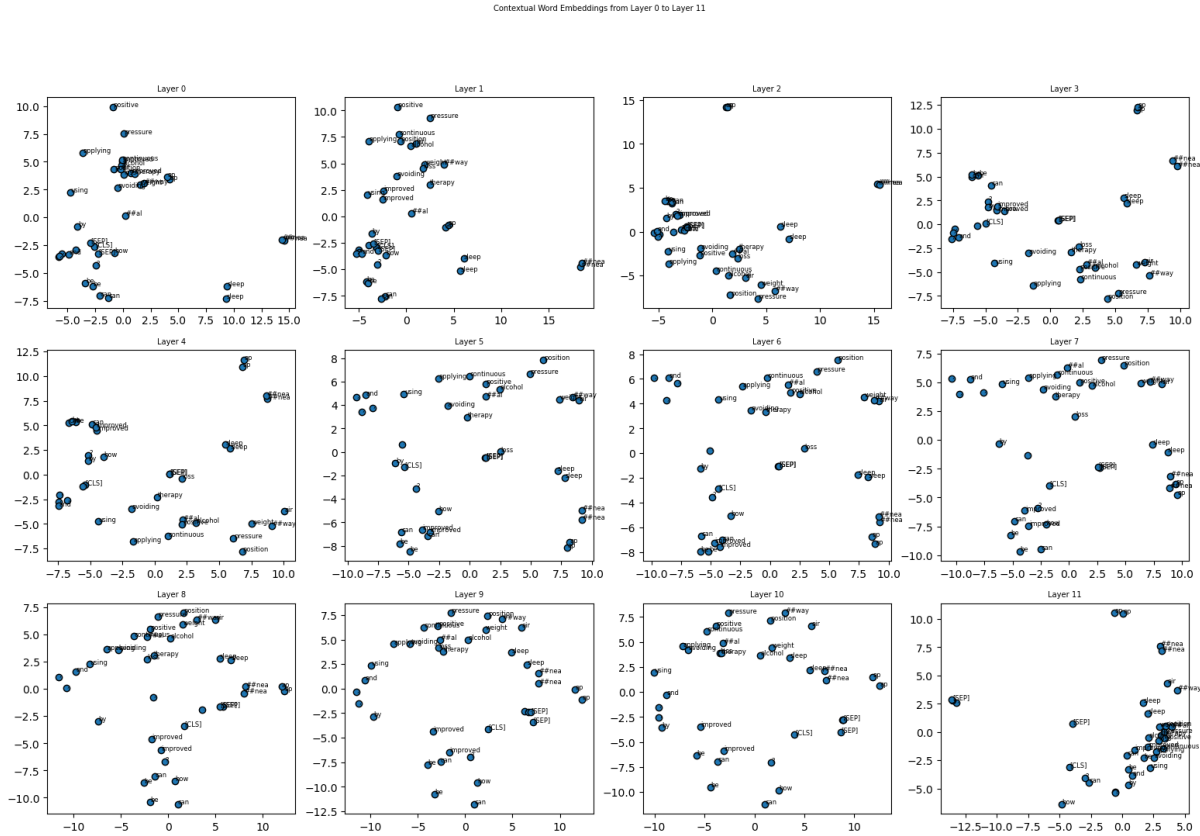


Figure 2: PCA projections of BERT token embeddings at each of the 12 transformer layers

Self-attention maps. Finally, we visualized the self-attention patterns of all 12 heads in layer 1. Each cell (i, j) shows how much word i attends to word j . Figure 3 shows all 12 heads at once.

The heads specialize in distinct roles. Heads 1 and 4 show a diagonal — each word attends only to itself, preserving token identity in early layers before deeper context builds up. Heads 5 and 9 route every word to [CLS], which the cross-encoder uses as the final relevance score. Head 10 is the most relevant for retrieval: query words attend directly to passage words, comparing question to evidence in a single forward pass — something a bi-encoder cannot do. Head 11 links the same word across sentences, connecting *sleep* in the query to *sleep* in the passage. Together

with the positional and contextual analyses, these patterns show how BERT builds from ordered token positions, through layer-by-layer context, to cross-sentence relevance judgments.

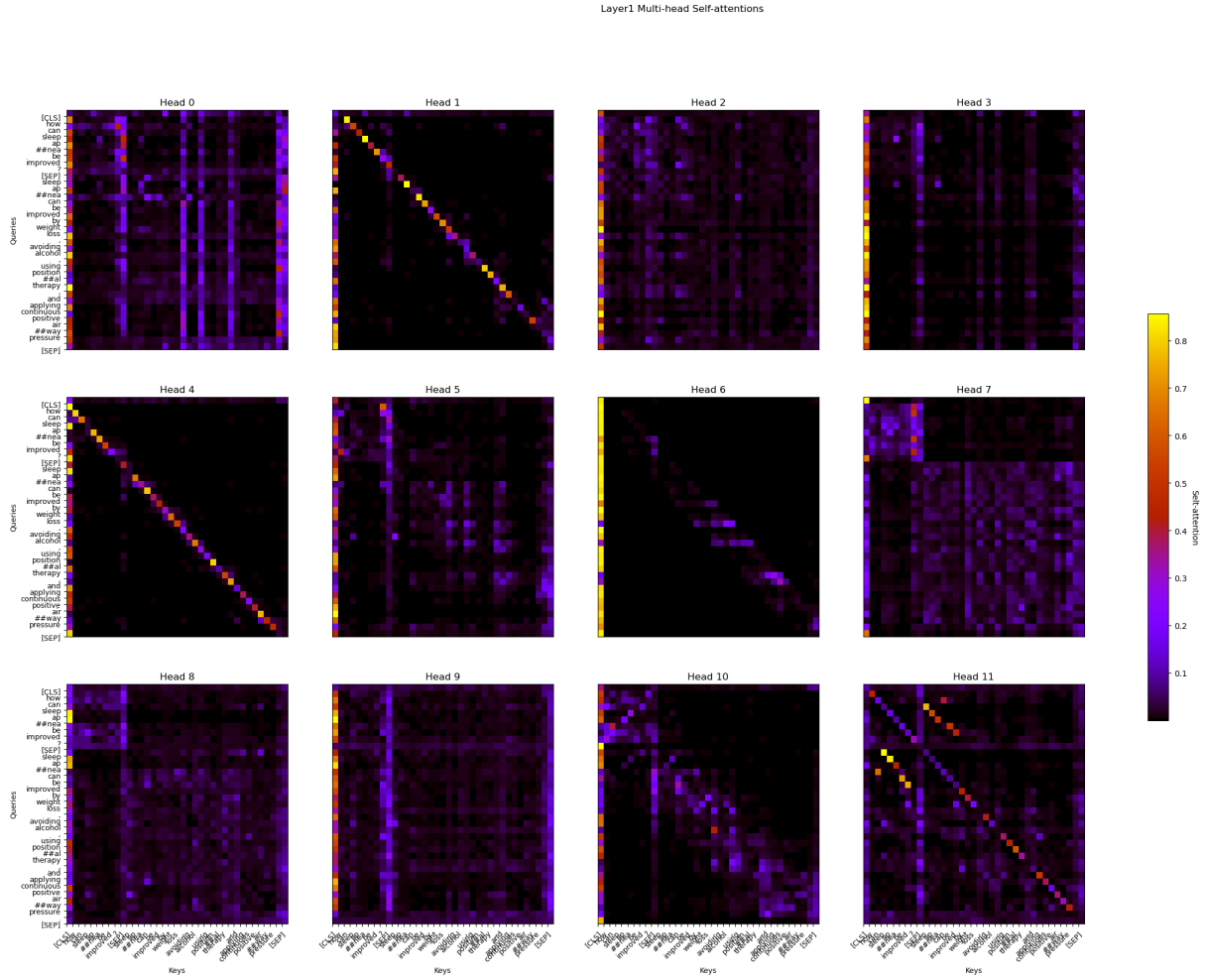


Figure 3: Self-attention matrices for all 12 heads in layer 1

2.3 LLM Agentic Patterns Phase 3

2.3.1 Overview

In Phase 3, the Phase 1 and Phase 2 components were extended into a biomedical Deep Research Agent. Instead of generating a single short answer directly from one retrieved evidence set, the agent follows an agentic pipeline that decomposes the original patient question into subtopics, retrieves evidence for each subtopic, and synthesizes a structured report.

The implemented agent follows an ADaPT-inspired pattern with four main stages:

1. **Plan**: generate focused biomedical subtopics from the original question;
2. **Browse**: run the Phase 1 retrieval and Phase 2 sentence-selection pipeline for each subtopic;
3. **Synthesize**: aggregate the evidence and generate a structured report;
4. **Evaluate**: use an LLM-as-a-judge to assess factual support, citation correctness, completeness, and safety.

2.3.2 Planning

The planning step uses a decoder LLM to decompose each patient question into focused subtopics. Each subtopic contains a short title, a search query, and a rationale explaining why the subtopic is useful for the final report. For example, a question about natural treatments for sleep apnea may be decomposed into subtopics such as lifestyle interventions, weight management, positional therapy, complementary approaches, and limitations of the evidence.

This step makes the system more exploratory than the Phase 2 RAG pipeline. Instead of retrieving evidence only for the original query, the agent creates several targeted retrieval queries, allowing the final report to cover different aspects of the biomedical question.

2.3.3 Subtopic Exploration

For each generated subtopic, the system executes a retrieval and evidence-selection cycle based on the previous phases. KNN retrieval from Phase 1 is used to retrieve the top PubMed articles for the subtopic query. The retrieved abstracts are then split into sentences and scored with the MedCPT cross-encoder, following the Phase 2 reference sentence selection strategy.

For Phase 3, each subtopic retrieves the top 3 documents and selects up to 2 reference sentences per document. This produces a compact evidence set for each subtopic while reducing the number of unnecessary sentences compared with selecting 3 sentences per article.

2.3.4 Evidence Aggregation

After all subtopics are explored, the selected evidence is aggregated into a single structured evidence object. Evidence is grouped by subtopic and duplicate PMID-sentence pairs are removed. Each evidence item stores the PMID, article title, selected sentence, retrieval score, and cross-encoder score.

This aggregation step allows the final generator to use evidence from multiple focused searches while still preserving traceability between each claim and its supporting PubMed article.

2.3.5 Report Generation

The final report is generated using the aggregated evidence. The generation prompt instructs the model to write a structured patient-facing biomedical report using only the provided evidence. The required report structure includes:

- a short answer;
- evidence organized by subtopic;
- limitations or uncertainty in the evidence;
- practical interpretation;
- a conclusion.

The prompt also requires every factual claim to include PMID citations and prevents the model from citing PMIDs that are not present in the retrieved evidence. This is important because

Phase 3 reports are longer than Phase 2 answers and therefore have a higher risk of unsupported generation.

2.3.6 Factual Consistency and Citation Validation

The Phase 3 system includes two evaluation mechanisms. First, an automatic citation validator extracts all PMIDs cited in the generated report and checks whether they belong to the set of PMIDs retrieved for that query. This detects citation hallucinations or invalid citation formats.

Second, GPT-4o-mini through the IAedu API is used as an LLM-as-a-judge. The judge evaluates each final report according to four criteria:

- **Factual support:** whether the main medical claims are supported by the retrieved evidence;
- **Citation correctness:** whether citations refer to valid evidence and support the corresponding claims;
- **Completeness:** whether the report addresses the main aspects of the question;
- **Safety:** whether the report is cautious and avoids unsafe medical advice.

The Phase 3 outputs were saved to `phase3_plans.json`, `phase3_subtopic_evidence.json`, `phase3_reports.json`, and `phase3_judge_results.json`.

3 Evaluation

3.1 Experimental Setup

The experiments use a PubMed corpus of 4,194 documents and 33 test queries with graded relevance judgments. Models were assessed with P@10, R@100, NDCG@10, NDCG@100, MAP, and MRR; precision-recall curves were also generated. All reported results are on the test split.

3.2 Results and Discussion

3.2.1 Main Quantitative Results

Table 1 presents the main retrieval results on the test set.

Table 1: Phase 1 test set retrieval results.					
Method	P@10	R@100	NDCG@10	NDCG@100	MAP
BM25	0.6667	0.8435	0.6417	0.7652	0.5979
JM	0.6758	0.8080	0.6503	0.7404	0.5693
LMD	0.6636	0.8104	0.6315	0.7374	0.5659
KNN	0.7485	0.8946	0.7207	0.8247	0.6719

3.2.2 Discussion

KNN achieved the best results across all metrics, capturing semantic similarity even when lexical overlap is limited. Among lexical methods, BM25 was the strongest baseline with competitive NDCG and the highest MRR. JM and LMD were weaker under default settings but improved after parameter tuning (Section 3.3). KNN is therefore the best candidate for Phase 2.

3.2.3 Precision-Recall Analysis

Figure 4 includes the precision-recall curves of the four retrieval methods.

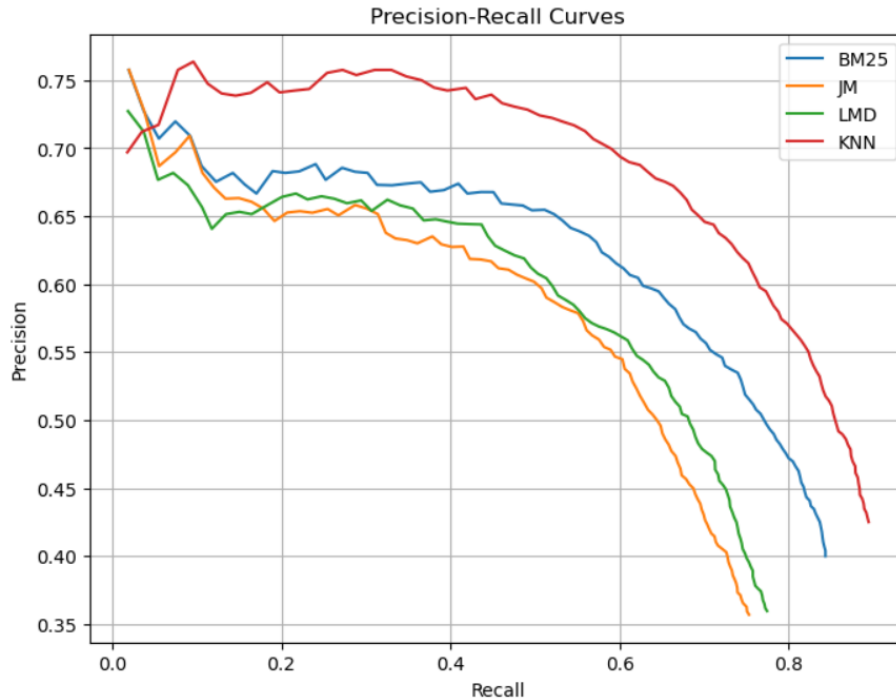


Figure 4: Precision-recall curves for the evaluated retrieval methods.

The precision-recall curves are expected to reflect the same overall tendency observed in the table, with KNN maintaining higher precision across a wider recall range than the lexical baselines.

3.2.4 Query-Level Precision-Recall Analysis

To further analyze system stability, we examine three representative queries: the query with the highest Average Precision (AP), the query with the lowest AP, and a query with median performance.

Figure 5 presents the precision-recall curves for these three queries using the KNN retrieval model.

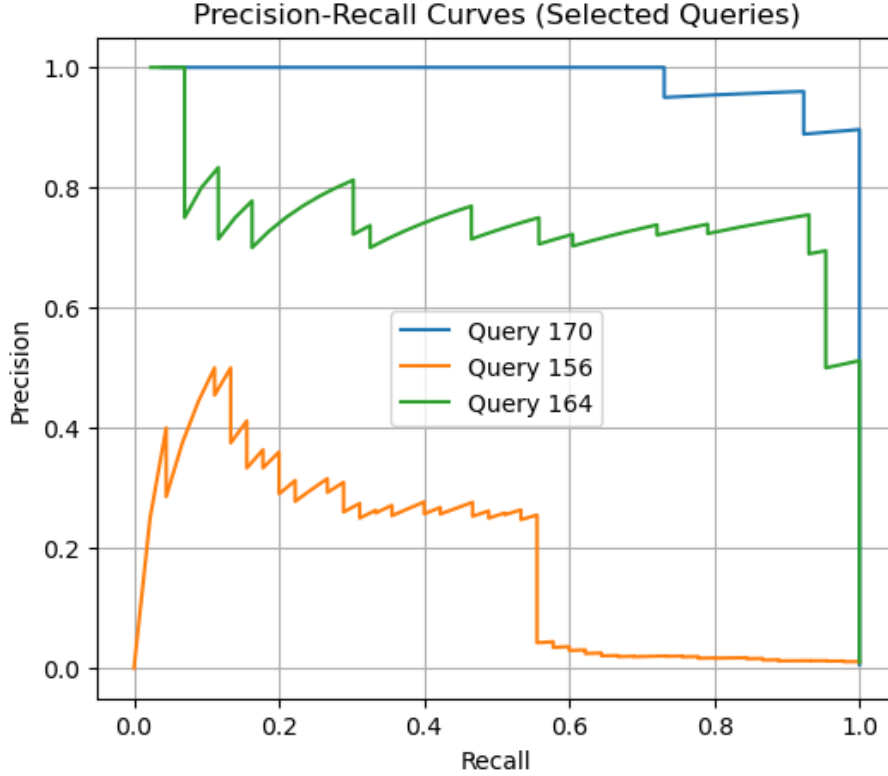


Figure 5: Precision-recall curves for selected queries (best, worst, and median AP).

Topic **170** (best AP) maintains near-perfect precision across most of the recall range, while Topic **156** (worst AP) drops to near-zero precision quickly. Topic **164** shows intermediate behavior with a gradual decline, consistent with vocabulary mismatch being the main failure mode for harder queries.

3.3 Hyperparameter Analysis

The default OpenSearch smoothing parameters ($\lambda = 0.1$, $\mu = 2000$) apply very heavy corpus smoothing. Given our small corpus of 4,194 documents with an average abstract length of 139 words, this is poorly suited — the corpus background is not reliable enough to dominate scoring. We therefore evaluated two alternative configurations on the training split: $\lambda = 0.2/\mu = 150$ and $\lambda = 0.5/\mu = 150$, setting μ near the average document length to balance document and corpus contributions.

Table 2 compares the default and best-tuned configurations for JM and LMD (KNN and BM25 are parameter-free).

Table 2: Default vs. best-tuned smoothing parameters (JM and LMD only).

Method	Config	MAP	NDCG@10	NDCG@100
JM	default ($\lambda = 0.1$)	0.5263	0.6309	0.6976
JM	tuned ($\lambda = 0.5, \mu = 150$)	0.5693	0.6503	0.7404
LMD	default ($\mu = 2000$)	0.5353	0.6144	0.7076
LMD	tuned ($\mu = 150$)	0.5659	0.6315	0.7374

Both methods improve consistently when μ is set near the average document length (139 words), yielding NDCG@10 gains of +3.1% for JM and +2.8% for LMD over the defaults.

3.4 Phase 2 Evaluation

3.4.1 Experimental Setup

The Phase 2 evaluation ran on all 33 test queries using the KNN outputs from Phase 1. Up to 3 reference sentences per retrieved article were selected by MedCPT, yielding 256 judged sentences (5–9 per query). Answers were generated with `google/medgemma-4b-it` and evaluated with GPT-4o through the IAedu API.

3.4.2 Reference Sentence Alignment Results

Table 3 summarizes the LLM-judge labels assigned to the 256 selected reference sentences.

Table 3: LLM-judge reference sentence alignment results over all 33 test queries.

Label	Count	Percentage
Required	60	23.4%
Borderline	42	16.4%
Unnecessary	150	58.6%
Inappropriate	4	1.6%
Total	256	100.0%

The results show that most selected sentences were judged as *Unnecessary*. This does not necessarily mean that the retrieval pipeline failed. The cross-encoder ranks sentences according to query-sentence semantic similarity, while the judge evaluates whether each sentence is necessary for a complete final answer. These are related but not identical objectives. A sentence can be highly related to the topic while still being too general, too methodological, redundant, or not essential for answering the patient’s specific question.

The relatively high number of *Unnecessary* labels therefore indicates that the sentence selection stage has high recall but limited precision. It often retrieves medically related evidence, but also includes background information, study objectives, prevalence statements, or general clinical context that is not needed in the final answer.

The *Required* and *Borderline* labels together account for 102 sentences, or 39.8% of all selected evidence. This suggests that the pipeline usually retrieves at least some useful supporting evidence, but the evidence set could be made more focused. A possible improvement would be to reduce the number of selected sentences per article, for example from 3 to 2, or to rank all candidate sentences globally and select only the top reference sentences across all retrieved documents instead of forcing the same number of sentences per article. We attempted to evaluate both alternatives, but the API became unstable during these runs and returned processing errors. The issue also occurred when rerunning the original configuration with the same parameters, suggesting that the failures were related to API availability rather than to the modified selection strategies. For this reason, we report the best complete set of results that we were able to obtain.

Only 4 sentences were labeled *Inappropriate*. These cases generally occurred when the sentence was medically related but potentially misleading or harmful in the context of the patient’s question. For example, evidence about serious complications or severe disease may be inappropriate when the question asks about a minor condition, because it may overstate the risk or fail to answer the intended question.

3.4.3 Answer Entailment Results

Table 4 summarizes the entailment labels assigned to the 33 generated answers.

Table 4: LLM-judge entailment results for generated answers over all 33 test queries.

Entailment Label	Count	Percentage
Supported	11	33.3%
Partially Supported	21	63.6%
Unsupported	1	3.0%
Total	33	100.0%

Most generated answers were judged as *Partially Supported*. This means that the generated answers were generally relevant and grounded in the selected references, but not completely supported by them. Only one answer was judged as *Unsupported*, which suggests that the RAG pipeline was usually able to prevent completely ungrounded generation.

The 11 *Supported* answers show that the system can generate well-grounded responses when the retrieved evidence directly matches the user’s information need. However, the dominance of *Partially Supported* labels reveals an important limitation. Some answers included details that were present in the retrieved references but not central to the question.

3.4.4 Qualitative Analysis

The Phase 2 results confirm that the overall pipeline is functional: documents retrieved by KNN are converted into sentence-level evidence, the MedCPT cross-encoder ranks the candidate evidence, MedGemma generates citation-bearing answers, and GPT-4o evaluates both evidence alignment and answer entailment.

Nevertheless, the experiment also shows a clear mismatch between cross-encoder relevance and judge necessity. Cross-encoder scores were often extremely high, even for sentences later labeled *Unnecessary*. This suggests that the cross-encoder is effective at identifying topical similarity, but its scores should not be interpreted as calibrated probabilities of answer usefulness. For patient-facing biomedical question answering, a sentence must not only mention related biomedical concepts; it must directly contribute to the answer.

A representative example is the corneal injury query. The selected evidence included the directly useful sentence stating that minor corneal abrasions usually heal within 48 hours, which was judged *Required*. However, other selected sentences described complications, deeper injuries, or general treatment approaches. Although medically related, these sentences were either unnecessary or potentially inappropriate because the patient was asking specifically about the healing time of a minor injury.

Another example is the sleep apnea query. Sentences about complementary and integrative approaches were useful, while sentences about conventional CPAP and oral appliances were judged unnecessary because the patient specifically asked about natural treatments. This shows that the pipeline retrieves semantically related medical evidence, but still needs stronger alignment with the intent expressed in the question and narrative.

3.4.5 Manual Inspection of Judge Labels

We manually inspected two examples from each sentence-level label to check whether the LLM-as-a-judge decisions were reasonable. Overall, the labels were mostly consistent with the prompt definitions.

For *Unnecessary*, the inspected sentences were topically related but did not directly answer the patient question. For example, a CBD sentence described the objective of a study rather than explaining whether CBD affects liver enzymes, and a corneal injury sentence discussed deeper corneal pathology rather than minor injury healing time.

For *Required*, the labels were appropriate because the sentences directly answered the question. The vocal cord paralysis example identified severe airway obstruction requiring emergency tracheostomy, and the spontaneous fracture example explained a relevant cause group for fractures without trauma.

For *Borderline*, the sentences were useful but incomplete. The Afrin/Sudafed example gave context about rhinitis medicamentosa but did not answer the drug-interaction question, while the tennis elbow example stated that the condition is self-limiting but did not provide a treatment recommendation.

For *Inappropriate*, the examples were medically related but potentially misleading or alarming in the specific patient context. For instance, severe corneal healing complications or facial numbness with sudden hemiparesis may be important warning signs, but they do not directly answer the patient’s original question.

This inspection suggests that the judge labels are mostly coherent. It also confirms that high cross-encoder scores do not always imply answer usefulness, since some *Unnecessary* and *Inappropriate* sentences still received scores close to 1.0.

3.4.6 Limitations and Possible Improvements

The main limitation of the current Phase 2 system is that reference sentence selection is performed independently inside each retrieved article. Since the system selects up to 3 sentences per document, it may keep weak sentences from a lower-quality article while excluding stronger sentences from another article. A better strategy would be to collect all candidate sentences from the top retrieved documents, rank them globally with the cross-encoder, and keep only the top 5 or 6 sentences overall. As discussed above, we attempted to evaluate this alternative, but the API became unstable during the run and returned processing errors.

A second limitation is that the current cross-encoder scores are not calibrated. Many selected sentences receive scores close to 1.0, including sentences later judged unnecessary. Future versions should therefore combine cross-encoder ranking with additional filtering rules, such as removing study-objective sentences, overly generic background sentences, duplicate sentences, and sentences with incomplete biomedical claims.

Overall, the Phase 2 results show that the system is able to generate mostly grounded biomedical answers, with 32 out of 33 answers judged either *Supported* or *Partially Supported*. However, the large proportion of *Unnecessary* reference sentences indicates that the evidence selection stage should be made more selective before the system is used as the basis for the Phase 3 biomedical deep research agent.

3.5 Phase 3 Evaluation

3.5.1 Experimental Setup

The Phase 3 evaluation was performed on the test queries using the Deep Research Agent pipeline. For each query, the system first generated a set of biomedical subtopics. Each subtopic was then processed independently through the KNN retrieval and MedCPT sentence-selection pipeline. The resulting evidence was aggregated and used to generate a structured biomedical report.

The generated reports were evaluated with GPT-4o-mini through the IAedu API. Unlike Phase 2, where the judge assigned a single entailment label to a short answer, Phase 3 uses four evaluation dimensions: factual support, citation correctness, completeness, and safety. An additional automatic citation validator was used to verify whether the PMIDs cited in each report were present in the retrieved evidence.

Due to IAedu processing limits during the judge evaluation stage, 32 reports were successfully judged. During development, the IAedu judge endpoint became unstable: rerunning the same code and prompts that had previously worked sometimes produced processing or rate-limit errors and later the endpoint began returning API URL errors. The professor suggested trying an alternative judge model, Qwen/Qwen3.6-27B, available through the NovaSearch endpoint. However, because the original Phase 3 experiments had already been completed earlier and the remaining period coincided with other course assessments, we did not have enough time to rerun and validate the full judge evaluation with the alternative model. Therefore, we report the 32 complete judge evaluations obtained with the IAedu GPT-4o-mini judge and explicitly note this as an evaluation limitation.

3.5.2 LLM-as-a-Judge Results

Table 5 summarizes the Phase 3 LLM-as-a-judge results.

Table 5: Phase 3 LLM-as-a-judge results over 32 judged reports.

Criterion	Label	Count	Percentage
Factual support	Supported	27	84.4%
Factual support	Partially Supported	5	15.6%
Citation correctness	Correct	32	100.0%
Completeness	Complete	24	75.0%
Completeness	Partially Complete	8	25.0%
Safety	Safe	32	100.0%

The results show that most generated reports were judged as factually supported. In total, 27 out of 32 reports were labeled *Supported*, while the remaining 5 were labeled *Partially Supported*. No report was judged as *Unsupported*. This suggests that the Deep Research Agent was generally able to synthesize longer answers while remaining grounded in the retrieved PubMed evidence.

Citation correctness was the strongest dimension, with all 32 judged reports labeled *Correct*. This indicates that the report generation prompt and citation validation step were effective at constraining the model to use PMIDs from the retrieved evidence set.

Completeness was slightly weaker. Although 24 reports were labeled *Complete*, 8 were only *Partially Complete*. This is expected in a deep research setting because some biomedical questions require broader evidence than what is available in the top retrieved documents for each subtopic. In some cases, the generated report answered the main question but missed secondary aspects such as long-term outcomes, safety considerations, or comparisons between alternative treatments.

All 32 judged reports were labeled *Safe*. This suggests that the model generally avoided unsafe medical instructions and used cautious language when the evidence was limited.

3.5.3 Citation Validation

In addition to the LLM judge, we performed automatic citation validation. The validator extracts PMIDs from the generated report and checks whether each cited PMID exists in the aggregated evidence for that query. This step is stricter than the LLM judge because it checks exact citation membership rather than semantic correctness.

During development, the first citation validator only recognized citations in the exact format [PMID:id]. However, the generated reports sometimes used grouped citations such as [PMID: 123, PMID: 456]. The validator was therefore updated to recognize flexible PMID citation formats, including grouped citations and citations with spaces after the colon. This correction ensured that valid grouped citations were not incorrectly counted as missing.

3.5.4 Qualitative Analysis

The Phase 3 results show that the agentic pipeline improves the structure and breadth of the generated outputs compared with Phase 2. By decomposing a question into subtopics, the system can cover several dimensions of a biomedical topic instead of producing a single short answer. For example, treatment-related questions can be organized into sections on first-line treatment, alternative approaches, risks, limitations, and practical interpretation.

The main advantage of this approach is that the final report is easier to inspect and more informative for complex biomedical questions. Grouping evidence by subtopic also makes the reasoning process more transparent: the reader can see which evidence was used for each part of the report.

However, the pipeline still depends strongly on the quality of the generated subtopics. If the planner produces subtopics that are too broad, repetitive, or slightly misaligned with the patient question, the retrieval stage may collect evidence that is medically related but not directly useful. This is similar to the limitation observed in Phase 2, where high cross-encoder similarity did not always imply answer usefulness.

3.5.5 Limitations and Possible Improvements

The main limitation of Phase 3 is that planning, retrieval, and synthesis errors can accumulate. A weak subtopic can lead to weak retrieved evidence, which can then affect the final report. Although the LLM judge results were strong, the system would benefit from an additional filtering or critique step after subtopic generation.

A second limitation is that the final reports are longer than Phase 2 answers, which increases the risk of unsupported details. The citation validator helps detect invalid PMIDs, but it does not prove that each cited sentence fully entails the claim. Future work should evaluate claim-level entailment, where each sentence of the final report is checked against the exact supporting evidence.

A third limitation is API stability. As in Phase 2, some IAedu calls returned processing or rate-limit errors. For this reason, the system includes retry logic and saves intermediate results after each successful call, allowing the evaluation to resume without losing progress.

Overall, Phase 3 shows that the BioGen NL Agent can be extended from short grounded answer generation to structured biomedical report generation. The results indicate that the agentic decomposition strategy improves coverage while maintaining factual grounding, but future work should strengthen subtopic quality control and claim-level citation verification.

4 Conclusion

4.1 Achievements

Phase 1 established the retrieval layer of the BioGen NL Agent. A PubMed corpus was indexed in OpenSearch and evaluated using BM25, JM, LMD, and KNN retrieval. KNN achieved the best results on all main test metrics, including P@10, R@100, NDCG@10, NDCG@100, and MAP.

BM25 remained the strongest lexical baseline, while JM and LMD improved after smoothing parameter tuning but did not surpass dense retrieval.

Phase 2 extended the retrieval system into a factually grounded RAG pipeline. MedCPT was used to select sentence-level evidence, MedGemma generated citation-bearing answers, and GPT-4o evaluated both reference sentence alignment and answer entailment. Of the 256 selected sentences, 102 were labeled *Required* or *Borderline*. At the answer level, 32 out of 33 generated answers were judged either *Supported* or *Partially Supported*, showing that the RAG system generally remained grounded in the retrieved PubMed evidence.

The BERT visualization exercise also helped illustrate the mechanisms behind transformer-based relevance models. The positional embedding analysis showed that BERT encodes token position continuously, the contextual embedding analysis showed that representations become increasingly context-dependent across layers, and the self-attention maps showed how different heads specialize in token identity, global summary routing, and query-passage interaction.

Phase 3 extended the system into a biomedical Deep Research Agent. The implemented ADaPT-inspired pipeline decomposes each question into subtopics, retrieves and selects evidence for each subtopic, aggregates the evidence, and synthesizes a structured long-form report. The Phase 3 evaluation showed strong results: 27 out of 32 judged reports were labeled *Supported*, all judged reports were labeled citation-correct and safe, and 24 reports were labeled complete. This indicates that the agentic pipeline can produce broader and more structured biomedical answers while maintaining factual grounding.

Overall, the three phases establish a complete biomedical natural language agent: document retrieval, sentence-level evidence selection, grounded answer generation, agentic subtopic exploration, structured report synthesis, and LLM-as-a-judge evaluation.

4.2 Limitations

Despite these achievements, the current system still has several limitations. In Phase 1, retrieval was evaluated using a single query formulation based on the concatenation of topic, question, and narrative. Other query formulations, query expansion techniques, or hybrid dense-lexical retrieval may improve retrieval effectiveness.

In Phase 2, the main limitation was the precision of reference sentence selection. Although the cross-encoder retrieved many topically related sentences, 150 out of 256 selected sentences were labeled *Unnecessary*. This shows that semantic similarity alone is not sufficient for selecting evidence for patient-facing biomedical answers. Some selected sentences were background statements, study objectives, duplicated evidence, or medically related details that did not directly answer the patient question.

In Phase 3, the main limitation is that errors can accumulate across the agentic pipeline. If the planner generates weak or overly broad subtopics, the retrieval step may collect evidence that is related but not central to the final answer. Similarly, longer reports increase the risk of including partially supported details. Although the citation validator checks whether cited PMIDs belong to the retrieved evidence set, it does not verify claim-level entailment for every sentence.

Another limitation across Phases 2 and 3 was API stability. Some IAedu calls returned processing or rate-limit errors, and later the endpoint began returning API URL errors, even when rerunning the same code and parameters that had previously worked. This limited the ability to test every alternative configuration and explains why the Phase 3 judge evaluation contains 32 out of 33 reports. The professor suggested trying Qwen/Qwen3.6-27B through the NovaSearch endpoint as an alternative judge model, but there was not enough time to fully rerun the evaluation with this model after the previous testing and debugging cycles. To reduce the impact of API instability, intermediate outputs were saved to JSON files and the judge evaluation was implemented with retry and resume support.

Future work should improve evidence filtering, test global sentence ranking instead of per-document sentence selection, add claim-level citation verification, and explore stronger agentic patterns with self-critique or iterative evidence refinement.

A Use of AI Assistants

ChatGPT was used to assist with debugging and revising parts of the implementation, mainly related to the IAedu API judge calls, response parsing, retry logic, handling of processing or rate-limit errors, and citation validation.

B Prompts Used with AI Assistants

The following prompts summarize the main types of assistance requested:

- Help debug the IAedu API judge call, including handling streaming responses, processing errors, and rate-limit errors.
- Help write retry and resume logic so that judge evaluation results could be saved incrementally and rerun without losing progress.
- Help parse IAedu token-streamed JSON responses for the Phase 3 judge output.
- Help fix citation validation so that grouped PMID citations such as [PMID: 123, PMID: 456] are correctly recognized.